

MY DECISION & RATIONALE

Chosen Stack: React + Vite + React Compiler + TypeScript (Frontend) + NestJS + TypeScript + Prisma + PostgreSQL (Backend)

I'm choosing this full-stack stack because **the portfolio itself needs to be dynamic** — and more importantly, **the platform is the proof**.

Why this stack:

- The platform is the portfolio:** My proof statement is "Video editing for Ubuntu. No NVIDIA. No rebooting." The best way to prove this is with a working, deployable application. Screenshots and text are secondary — the live demo is primary.
- System design is already done:** The repository shows I've already created system design artifacts: Component Diagram, Context-Level DFD, ERD, System Architecture, Use Case Diagram, and Workflow/State Diagram. This means the architecture is planned — I'm not building blind.
- Full-stack experience:** I have 10 years of experience. I can maintain this stack comfortably.
- Free hosting exists:** Netlify/Vercel for the frontend, Render/Railway for the backend, Supabase/Neon for PostgreSQL. All have free tiers that work for a portfolio project.
- Shows my work properly:** A visitor can actually use the platform. That's more convincing than any case study write-up.

Why Not Option 1 (Plain HTML + GitHub Pages)

It can't show a dynamic, full-stack application. The portfolio would just be a brochure — not a working demo. For a project that's a working platform, this is insufficient.

Why Not Option 2 (Astro + Netlify)

Same problem — it's static. Even with Markdown case studies, it can't embed a working backend. The platform needs to be live and interactive to prove my claim. A

static site with screenshots is a step up from plain HTML, but it still doesn't show the real thing.

Can I Maintain This?

Yes — and this is the most important question. I've worked with React, TypeScript, and Node.js for years. NestJS and Prisma are new but follow familiar patterns. PostgreSQL is a standard database. The maintenance burden is real (dependency updates, security patches, database migrations) but manageable for a solo developer with my experience level.

Backend Question: Yes, It Needs a Backend

The platform itself is a full-stack video editing application. It needs a backend for:

- User authentication (if added)
- Video processing (even if just metadata)
- Storing project data
- Exporting videos

The portfolio doesn't just describe the backend — it is the backend. This is a key difference from a typical portfolio. I'm not building a site about the platform; I'm building the platform itself, and the portfolio is the frontend that showcases it.